

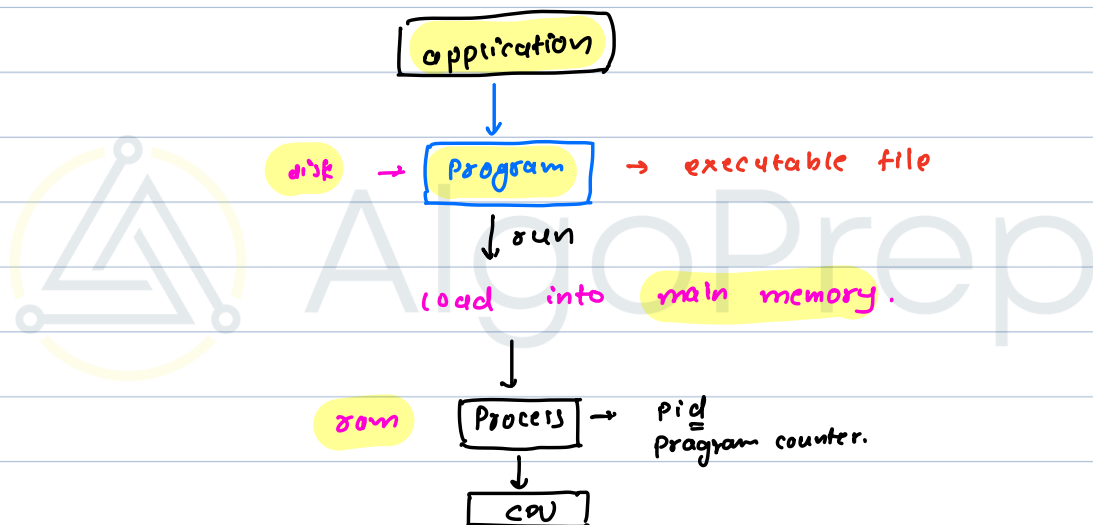
Good evening everyone



class start at 8:10

thread

How process will be created?



ms word

hey, my name is nitesh.

spell check.

auto save

suggestion

grammar check

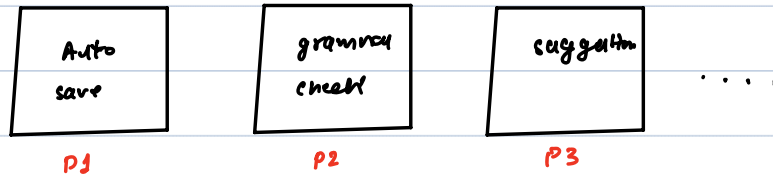
formatting.

pointer blink.

⋮
so on

we want all the features execute at same time.

CPU execute 1 process



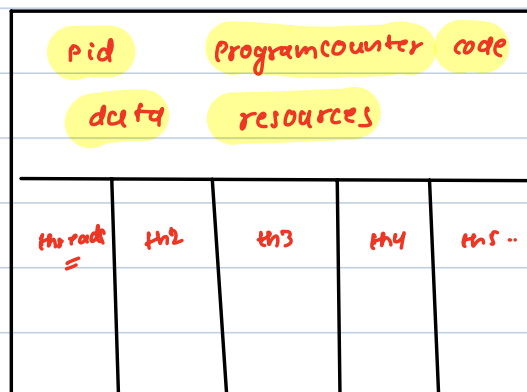
Process



↳ multiple threaded environment.

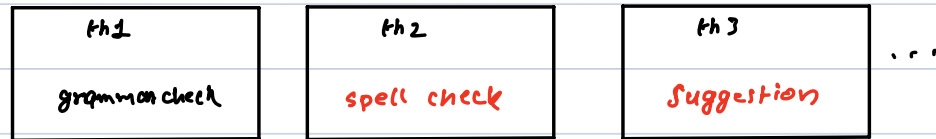
each process have one or many threads.

Process



• A thread is sub unit of a process.

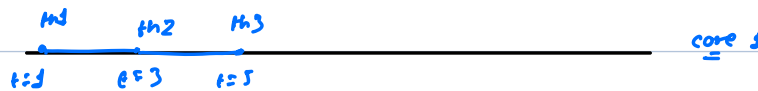
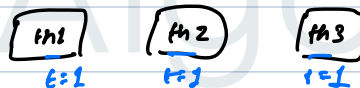
- it represents the smallest unit of execution within the process.



single core vs multi core processor.

Single core processor.

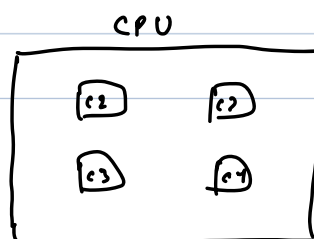
it consist only one processing unit (core)



we can achieve concurrency by executing all thread.

multi core processor.

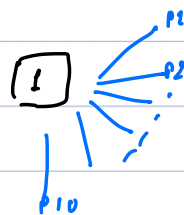
multi core processor have two or more processing unit (cores).



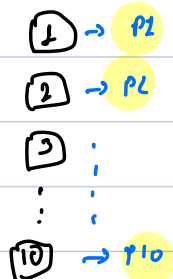


we can achieve true parallelism in multi core processor.

10 people golang



concurrency



Parallelism

CPU

↳ gamers

if contains 100 to 1000 cores

↓

1000 threads can run at same time.

machine learning

8 cores → 50K

1000 cores → $\frac{50}{8} \times 1000$

to achieve parallelism.

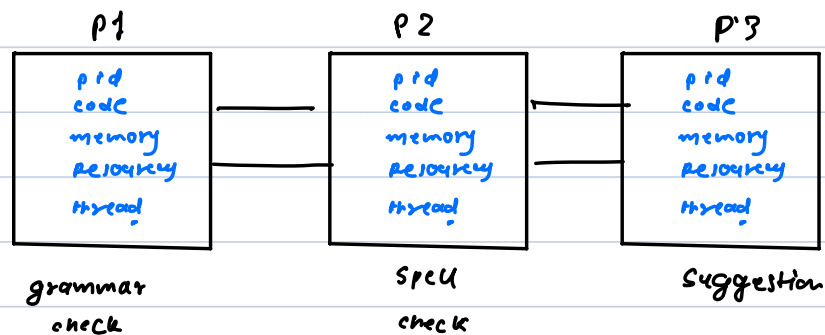
CPU

2.5 GHz - 5 GHz

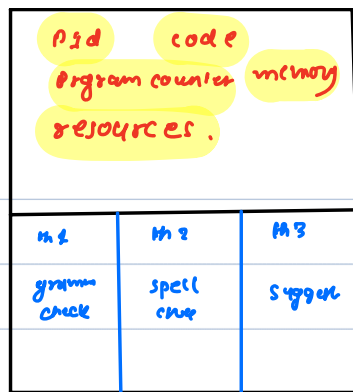
CPU

0.5 GHz ... 2 GHz

Process vs Thread

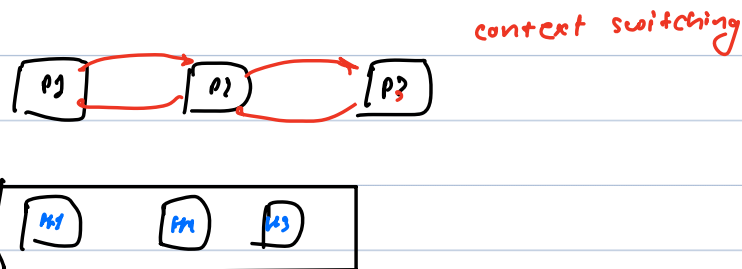


process



- each process has separate memory space.
- all threads within the process share the same memory space and other resources.

- process takes more storage as compared to thread.



- higher overhead due to separate memory space and context switching in process, and lower overhead in thread.

communication

Threads can communicate directly through shared memory.

we use inter process communication (expensive)
to communicate between process.

break till 9:10

In java we have atleast one thread
(main thread)

- to get the current thread name

```
Thread.currentThread().getName();
```

there are two method to create thread.

- 1) to extend the thread class.
- 2) to implement the runnable interface.

method : define the task

- create a subclass to extend Thread class.
- override the run() method.
- create an object of that thread class.
- start the thread using start() method.

meth2

• define the task.

• create a subclass by implement Runnable interface.

```
class myThread1 implement Runnable {  
    override.  
    public void run() {
```

```
    }
```

```
}
```

* create an object of myThread1.

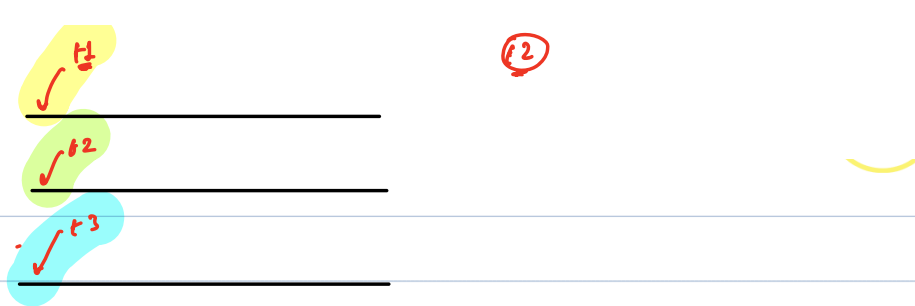
```
myThread1 task1 = new myThread1();  
Thread t1 = new Thread(task1);  
t1.start();
```

Q) print number from 1 to 100 with separate threads.

threads are executing randomly.
there is no order.

Q) print number from 1 to 100 with separate threads in order.

wait to complete the previous thread.
join()



```

1
2 class thread5 extends Thread{
3
4     //create constructor
5     private int number;    //1 , 2, 3 ,..100
6     public thread5(int num){
7         this.number = num;
8     }
9
10    public void run(){
11        System.out.println("my task is :" + number + Thread.currentThread().getName());
12    }
13
14 }
15
16 public class task1 {
17     Run | Debug
18     public static void main(String[] args) {
19         //print 1 to 100 using seperate threads
20         for(int i = 1; i<=100; i++){
21             thread5 t5 = new thread5(i);
22             t5.start();
23         }
24
25         System.out.println(x:"this is end");
26
27     }
28 }
29
30
31
32 public class impThread {
33     Run | Debug
34     public static void main(String[] args) {
35         System.out.println(x:"Hello world");
36         System.out.println("curr thread : " + Thread.currentThread().getName());
37         // fun1();
38
39         // mythread1 t1 = new mythread1();
40         // t1.start();
41         // mythread2 t2 = new mythread2();
42         // t2.start();
43         mythread3 task3 = new mythread3();
44         Thread t3 = new Thread(task3);
45         t3.start();
46     }
47
48     public static void fun1(){
49         System.out.println(x:"i am fun1");
50         System.out.println("curr thread2 : " + Thread.currentThread().getName());
51     }
52 }
53

```

```
1  class mythread1 extends Thread{
2
3      public void run(){
4          int i = 100;
5          while(i-->0){
6              System.out.println(x:"task 1 is executing");
7              System.out.println("curr thread1 : " + Thread.currentThread().getName());
8          }
9      }
10 }
11
12 class mythread2 extends Thread{
13     public void run(){
14         int i = 100;
15         while(i-->0){
16             System.out.println(x:"task 2 is executing");
17             System.out.println("curr thread2 : " + Thread.currentThread().getName());
18         }
19     }
20 }
21
22 class mythread3 implements Runnable {
23     public void run(){
24         int i = 100;
25         while(i-->0){
26             System.out.println(x:"task 3 is executing");
27             System.out.println("curr thread3 : " + Thread.currentThread().getName());
28         }
29     }
30 }
```